SkillMatch — AI-Powered Internship & Career Recommendation Platform

> *A production-quality full-stack web application designed to bridge the gap between university students and industry internship opportunities through deterministic matching algorithms and explainable AI guidance.*

🌟 Executive Overview

SkillMatch is a full-stack platform built with Next.js 14 (App Router), TypeScript, Tailwind CSS, Prisma ORM, and Auth.js (NextAuth v5). The platform allows students to showcase their academic history, skill inventory with proficiency levels, and career aspirations, while matching them against open internship listings.

Unlike generic job portals that rely on unpredictable LLM black-box ranking, SkillMatch features a **deterministic mathematical matching engine** (0–100% score) that ensures objective evaluation. An **abstracted AI service layer** enriches candidate recommendations with natural language *"Why You Match"* explanations, skill-gap diagnostic reports, and interactive AI career mentorship—operating 100% offline out-of-the-box via a built-in `MockAIProvider` or online via `OpenAIProvider`.

🚀 Key Features

1. Student Candidate Experience

- **Authentication & RBAC**: Secure sign up and sign in powered by Auth.js and encrypted JWT cookies with Edge-level route protection.
- **Profile & Skill Inventory Builder**:
  - Personal information and career preferences (Target Role, Preferred Work Mode: Remote/Hybrid/Onsite, Preferred Category).
  - Education history manager (University, Degree, Field of Study, Dates, GPA).
  - Interactive skill tagger with 4-tier proficiency levels (`BEGINNER` , `INTERMEDIATE` , `ADVANCED` , `EXPERT`).
  - Dynamic profile completion indicator (0% to 100%).
- **Public Internship Discovery**:
  - High-performance multi-filtering by keywords, category, location, and work mode.
  - Published-only visibility guard (draft/closed listings are hidden from public view).
- **Internship Detail & Express Application**:
  - Full specification view (responsibilities, qualifications, stipend, deadline).
  - Cover note application submission modal with duplicate submission prevention (`@@unique([studentId, internshipId])`).
- **Application Tracking Dashboard ("My Applications")**:
  - Live status pipeline tracker (`APPLIED` , `UNDER_REVIEW` , `INTERVIEWING` , `OFFERED` , `REJECTED` , `WITHDRAWN`).
  - Filter by status tabs and application withdrawal capability.

2. AI & Intelligence Layer

- **Deterministic Internship Match Engine**: Mathematically calculates candidate-job alignment (0–100%) based on skill overlaps, proficiency weighting, location proximity, and category match.
- **AI "Why You Match" Explanations**: Generates natural language match rationale based strictly on structured scoring breakdown.
- **AI Skill-Gap Diagnostic Reports**: Identifies matched skills vs missing required skills and generates actionable learning roadmaps.
- **Interactive AI Career Mentor**: Conversational chatbot (`/student/ai-guidance`) grounded in candidate profile context for technical resume optimization and interview prep.
- **Pluggable AI Architecture**: Abstracted interface (`IAIServiceProvider`) supporting `MockAIProvider` (offline zero-config demo mode) and `OpenAIProvider` (OpenAI API key), with automatic fallback resilience.

3. Administrator Management Portal

- **Admin Analytics Dashboard**: Real-time database metrics for registered candidates, active job listings, candidate status pipeline breakdowns, and top in-demand skills.
- **Internship Listing CRUD**: Create, edit, publish/unpublish (`PUBLISHED` , `DRAFT` , `CLOSED`), and delete job listings with skill requirements.
- **Category Taxonomy Management**: Category manager with slug generation and relational integrity protection (prevents deleting categories referenced by active listings).
- **Candidate Application Review Portal**: Review applicant profiles, inspect cover letters, education, and skills, and update pipeline statuses.

🛠️ Technology Stack

Layer	Technology
Framework	Next.js 14+ (App Router)
Language	TypeScript (Strict mode)
Styling	Tailwind CSS + Vanilla CSS Variables
Icons	Lucide React
Database	SQLite (Dev) / PostgreSQL (Prod ready)
ORM	Prisma ORM 5.x
Authentication	Auth.js (NextAuth.js v5) with bcryptjs password hashing
Validation	Zod
AI Integration	Pluggable Service Layer (Mock AI + OpenAI API architecture)

📐 Deterministic Matching Engine Formula

The primary numeric match score is calculated deterministically by application code in `src/lib/matching.ts` :

$$\text{Match Score} = (0.60 \times \text{SkillScore}) + (0.20 \times \text{LocationScore}) + (0.20 \times \text{CategoryScore})$$

Scoring Rules

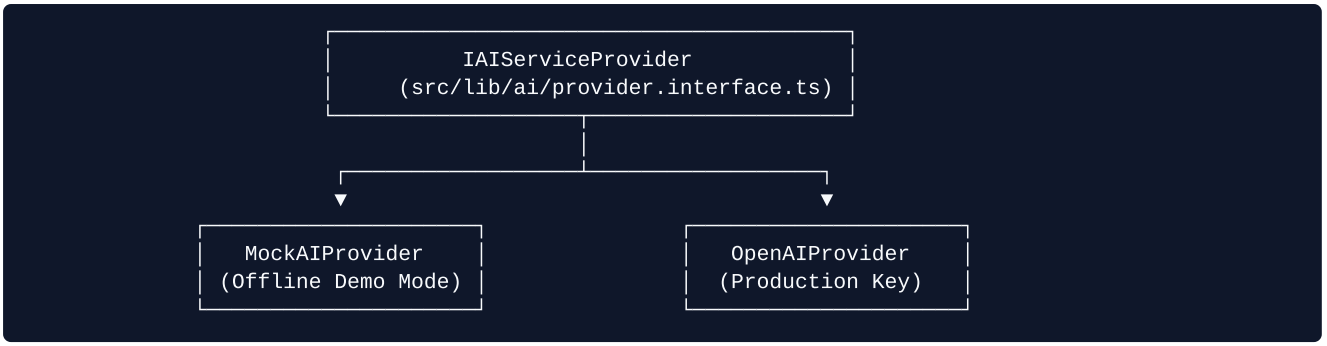
1. **SkillScore (60% Weight)**:

   $$\text{SkillScore} = [\ \sum_{\text{matching required skills}} (\text{ProficiencyWeight}) \ / \ (\text{Total Required Skills Count})\ ] \times 100$$

   Proficiency weights: `BEGINNER = 0.70` , `INTERMEDIATE = 0.85` , `ADVANCED = 0.95` , `EXPERT = 1.00` .
2. **LocationScore (20% Weight)**:
   - `100` points if job is 100% Remote, or student preferred work mode matches job work mode, or location substring matches.
   - `70` points for Hybrid jobs when candidate prefers Remote.
   - `40` points baseline for mismatched locations.
3. **CategoryScore (20% Weight)**:
   - `100` points for exact preferred category match.
   - `80` points for target role text alignment.
   - `50` points baseline for different domain categories.

> ⚠️ ***Note**: The LLM **never** determines or overrides the numeric match score. The match score is mathematically guaranteed to remain predictable, testable, and bounded between 0% and 100%.*

🤖 Abstract AI Service Architecture



The application uses an AI Factory (`ai.service.ts`). If no `OPENAI_API_KEY` is provided or if `AI_PROVIDER="mock"` , the platform operates using `MockAIProvider` . If an OpenAI request fails or times out, the service catches the exception and falls back to `MockAIProvider` so the application never crashes.

📁 Directory Structure

```
SkillMatch/
├── prisma/
│   ├── schema.prisma       # Database schema (SQLite / PostgreSQL compatible)
│   └── seed.js             # Database seeder (Admin, Student, Skills, Categories, Jobs)
├── src/
│   ├── app/
│   │   ├── (auth)/         # Login & Register routes
│   │   ├── (dashboard)/
│   │   │   ├── student/    # Student Dashboard, Profile, Applications, Recommendations, AI
Guidance
│   │   │   └── admin/      # Admin Overview, Internships CRUD, Applications, Categories
│   │   ├── internships/    # Public Job Discovery & Detail Specification views
│   │   ├── api/auth/       # Auth.js API route handler
│   │   ├── layout.tsx
│   │   └── page.tsx        # Landing Page
│   ├── components/
│   │   ├── ui/             # Reusable UI library (Button, Input, Card, Modal, Select, Badge,
Navbar)
│   │   ├── student/        # ProfileForm, EducationManager, SkillManager, ApplicationsList
│   │   ├── admin/          # InternshipTable, InternshipFormModal, CategoryManager,
ApplicationsTable
│   │   └── ai/             # AIRecommendationCard, AIChatBox, SkillGapModal
│   ├── lib/
│   │   ├── db.ts           # Prisma client singleton
│   │   ├── auth.ts         # Auth.js NextAuth v5 configuration
│   │   ├── matching.ts     # Deterministic matching engine
│   │   ├── ai/             # Abstract AI Provider architecture
│   │   └── actions/        # Server Actions (Auth, Profile, Admin CRUD, Applications, AI)
│   └── middleware.ts       # Edge Role-Based Access Control
├── .env.example
├── package.json
└── tsconfig.json
```

💻 Local Setup & Development Instructions

1. Prerequisites: Node.js 18.x or 20.x installed. npm or pnpm package manager.

2. Installation: `git clone https://github.com/your-username/SkillMatch.git && cd SkillMatch && npm install`

3. Environment Setup: Copy `.env.example` to `.env` with default DB and Auth secrets.

4. Database & Tests: Run `npm run db:push` , `npm run db:seed` , and unit tests via `node dist/matching.test.js` .

🔑 Demo Accounts for Technical Evaluation

Role	Email	Password	Description
Student	`student@skillmatch.com`	`StudentPassword123!`	Pre-populated profile (Stanford CS student, skills, education)
Admin	`admin@skillmatch.com`	`AdminPassword123!`	Platform Administrator (Full CRUD & Application Pipeline access)

> 💡 ***Tip**: Quick-fill buttons are provided directly on the `/login` page for evaluator convenience.*

🎭 Recommended Portfolio Demo Walkthrough

1. **Unauthenticated Visitor**:
   - Visit `/` landing page. View platform hero banner, features, and demo credentials box.
   - Click **Explore Internships** (`/internships`) to search open listings using keyword and category filters.
2. **Student Candidate Experience**:
   - Sign in as `student@skillmatch.com` .
   - Navigate to **Profile** (`/student/profile`): Edit personal bio, add an education record, and tag skills with proficiency levels (`BEGINNER` – `EXPERT`). Notice completion percentage update.
   - Navigate to **AI Recommendations** (`/student/recommendations`): View jobs ranked by deterministic match score (e.g. `92% Match`). Click *"Why You Match"* to expand AI rationale or *"Skill-Gap Analysis"* to view recommended learning roadmaps.
   - Open a job detail page (`/internships/[id]`), click **Apply for Position**, write an optional cover note, and submit.
   - Navigate to **My Applications** (`/student/applications`) to monitor the live application pipeline status.
   - Navigate to **AI Mentor** (`/student/ai-guidance`) and ask career advice questions.
3. **Admin Platform Manager**:
   - Sign in as `admin@skillmatch.com` .
   - Access **Admin Overview** (`/admin/dashboard`): Inspect candidate analytics, application pipeline stats, and top in-demand skills.
   - Access **Manage Jobs** (`/admin/internships`): Create a new job listing, edit existing details, or toggle publish/unpublish status.
   - Access **Applications** (`/admin/applications`): Inspect candidate profiles and progress application status to `INTERVIEWING` or `OFFERED` .
   - Log back in as Student to confirm the updated application status reflects in real-time.

🔒 Security Considerations

- **Server-Side Access Control**: Route protection enforced at Edge via `src/middleware.ts` . Server actions validate `session.user.role` before performing database mutations.
- **Database Safety**: Prepared queries via Prisma ORM eliminate SQL injection vulnerabilities.
- **Secret Isolation**: `OPENAI_API_KEY` and secret tokens reside strictly in server environment variables.
- **Relational Protection**: Category deletion safeguards prevent accidental cascading deletions of active job listings.

📋 Production Readiness Notes

For a full cloud deployment (e.g. Vercel + Managed PostgreSQL):

[x] Strict TypeScript & ESLint verification.
[x] Bounded deterministic matching score logic.
[x] Fallback AI provider architecture.
[ ] Connect managed PostgreSQL database URL (Neon / Supabase).